



MODUL 10 · VALIDASI DATA & FORMS

Validasi di Server Actions & API Routes



Validasi server adalah WAJIB — tidak bisa bergantung hanya pada validasi client.

Validasi di Server Action — Pattern Standar

Server Action menerima FormData. Selalu safeParse dengan Zod sebelum operasi DB.

```
"use server"
import { z } from "zod"
import { db } from "@/lib/db"
import { revalidateTag } from "next/cache"
import { createPostSchema } from "@/lib/validations/post"

type ActionResult<T> =
  | { success:true; data:T }
  | { success:false; errors:Record<string,string[]> }

export async function createPost(
  formData: FormData
): Promise<ActionResult<{ id:string; slug:string }>> {

  // 1. Buat object dari FormData
  const rawData = {
    title:    formData.get("title"),
    content:  formData.get("content"),
    categoryId:formData.get("categoryId"),
    published: formData.get("published") === "true",
  }

  // 2. Validasi dengan Zod — JANGAN gunakan parse()!
  const validated = createPostSchema.safeParse(rawData)
  if (!validated.success) {
    return {
      success: false,
      errors: validated.error.flatten().fieldErrors
        as Record<string, string[]>,
    }
  }

  // 3. Gunakan validated.data — sudah typed dan aman
  const post = await db.post.create({ data:validated.data })
  revalidateTag("posts")
  return { success:true, data:{ id:post.id, slug:post.slug }
}
```

Validasi di API Route Handler

Route Handler terima Request JSON. z.coerce untuk query params. Proper HTTP status codes.

```
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'
import { db } from '@lib/db'
import { createPostSchema } from '@lib/validations/post'

// z.coerce: konversi string URL params ke number
const querySchema = z.object({
  page: z.coerce.number().min(1).default(1),
  limit: z.coerce.number().min(1).max(50).default(10),
  q: z.string().optional(),
})

export async function GET(req: NextRequest) {
  const params = Object.fromEntries(req.nextUrl.searchParams)
  const query = querySchema.safeParse(params)
  if (!query.success) {
    return NextResponse.json(
      { errors: query.error.flatten().fieldErrors },
      { status: 400 }
    )
  }
  const { page, limit, q } = query.data // typed!
  const posts = await db.post.findMany({
    take: limit,
    skip: (page - 1) * limit,
    where: q ? { title: { contains: q, mode: 'insensitive' } } :
  {}))
  return NextResponse.json({ posts, page, limit })
}

export async function POST(req: NextRequest) {
  const body = await req.json().catch(() => null)
  const validated = createPostSchema.safeParse(body)
  if (!validated.success) {
    return NextResponse.json(
      { errors: validated.error.flatten().fieldErrors },
      { status: 422 }
    )
  }
  const post = await db.post.create({ data: validated.data })
  return NextResponse.json(post, { status: 201 })
}
```

Error Display — fieldErrors ke UI

.flatten().fieldErrors → Record<fieldName, string[]> — format ideal untuk display per field.

```
// Server Action return errors:
// { title: ["Min 3 karakter"],
//   content: ["Konten terlalu pendek"],
//   categoryId: ["Kategori tidak valid"] }

// Di form page — Server Component
export default async function NewPostPage() {
  const searchParams = await ..searchParams
  // atau dari useActionState state.errors

  return (
    <form action={createPost}>
      <div>
        <label htmlFor="title">Judul</label>
        <input id="title" name="title" />
        {/* Tampilkan error jika ada */}
        {errors?.title?.[0] && (
          <p className="text-red-500 text-sm">
            {errors.title[0]}
          </p>
        )}
      </div>
      <div>
        <label htmlFor="content">Konten</label>
        <textarea id="content" name="content" />
        {errors?.content?.[0] && (
          <p className="text-red-500 text-sm">
            {errors.content[0]}
          </p>
        )}
      </div>
      <button type="submit">Simpan
    </form>
  )
}
```

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — Mengapa Server Action harus validasi ulang meskipun client sudah validasi?

A) Untuk performa yang lebih baik

B) TypeScript tidak bekerja di client

C) Client validation bisa di-bypass — Server Action bisa dipanggil langsung tanpa UI

Q2 — Apa fungsi `z.coerce.number()` yang berbeda dari `z.number()`?

A) `z.coerce.number()` lebih strict

B) Hanya untuk PostgreSQL

C) `z.coerce.number()` konversi string '42' → number 42 dulu sebelum validate

Q3 — `Error.flatten().fieldErrors` menghasilkan format apa?

A) Array semua error messages

B) JSON string dari semua errors

C) `Record<string, string[]>` — key=nama field, value=array pesan error

Tugas Mandiri

Implementasikan validasi server-side yang lengkap.

01 Server Action

Buat createPost Server Action: extract FormData → safeParse → return {success:false, errors} jika gagal → db.create jika sukses.

02 API Route

GET /api/posts dengan z.coerce untuk page dan limit query params.
POST /api/posts dengan body validation. Status 400/422/201 yang tepat.

03 Error Display

Di form: tampilkan errors dari server action. errors?.title?.[0] di bawah input title. errors?.content?.[0] di bawah textarea.

04 Security Test

Test bypass dengan curl: curl -X POST /api/posts -d '{}'. Verifikasi: validasi server tetap berjalan dan reject data invalid.



Never trust the client. Server Action bisa dipanggil langsung tanpa form/UI apapun.

Yang Sudah Kita Pelajari

- ✓ Server Action: FormData → object → safeParse → check success → gunakan result.data.
- ✓ API Route: z.coerce.number() untuk query params string → number. Status 400/422/201.
- ✓ .flatten().fieldErrors: {fieldName: string[]} — format ideal untuk display per field.
- ✓ Never trust client — Server Action bisa dipanggil langsung tanpa form di browser.
- ✓ Satu schema, dua tempat: import schema yang sama di client dan server.

→ Selanjutnya: **Bab 4 — react-hook-form + Zod Resolver**

